

KISANPOOL

PROJECT OVERVIEW

1.1 Problem Statement

It is difficult for small-scale farmers to possess small quantities of produce and find transportation to the nearby market or mandi since transportation would be costly in that case. Some of the problems include:

- The cost of first mile transportation is very high.
- Small and dispersed farm loads.
- Vehicles remain underutilized.
- Returns trips remain empty.
- Bargaining power remains low.
- Trouble in finding transport.

1.2 Proposed Solution

KisanPool is an internet and mobile-based platform where farmers in proximity can avail themselves of transport services.

Farmers need to register their produce, quantity, pick-up point and destination. KisanPool clusters the requests that match and provides farmers with suitable transport facilities along with multiple stops pick up schedule and transportation costs.

Solution Flow

Farmers → Group Loads → Match Vehicle → Optimize Route → Mandi → Share Cost

1.3 Objectives

- Reduce transportation cost for small farmers
- Improve vehicle utilization
- Reduce unnecessary trips
- Simplify transport booking
- Provide transparent cost sharing
- Improve access to agricultural markets

1.4 Technical Overview

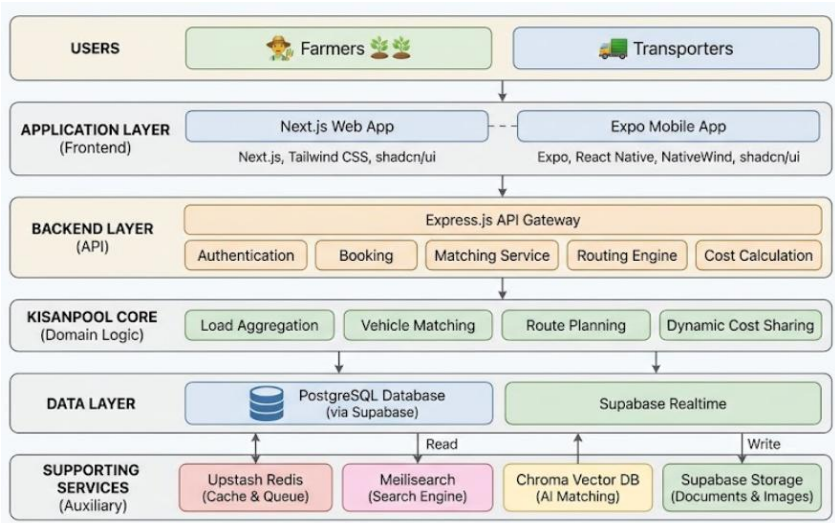
KisanPool is a Next.js web application and Expo mobile application that has an Express.js backend. The database used is PostgreSQL with Supabase.

Some of the technologies used in the project implementation are Upstash Redis, Meilisearch, Chroma, Supabase Storage, Docker Compose, and Render. Figure 1. Basic Solution Flow

SYSTEM ARCHITECTURE & TECHNOLOGY STACK

2.1 Architecture

KisanPool bridges farmers and transporters through both web and mobile apps. All the above tasks are executed on the server side, where the database stores all information.



Users: Farmers place transport orders; drivers offer transportation vehicles.

Application Layer: Websites and mobile applications for users.

Back-end: Handles authentication, booking, and routing processes.

Core Engine: Bunches loads, finds transport vehicles, routes, and divides cost sharing.

Data Layer: Stores and manages data within the application.

Support Services: Caching, searching, AI-based matching, and file storage.

Figure 1. System Architecture

2.2 Key Modules

- Authentication: User Registration, Login and Roles.
- Transport Request: Enables farmers to input their products, quantities, pick-up, and destination information.
- Vehicle Management: Enables transporters to control their vehicle capacity and availability.
- Matching: Matches farmers’ requests with available vehicles that fit their criteria.
- Routing: Generates an efficient route for making multiple stops.
- Booking and Cost: Handles trip booking and calculates the shared transportation cost.

2.3 Technology Stack

Layer	Technology
Web	Next.js
Mobile	Expo
UI	Tailwind CSS, shadcn/ui
Backend	Express.js
API	REST API
Database	PostgreSQL + Supabase
Authentication	Supabase Auth
Cache	Upstash Redis
Search	Meilisearch
Storage	Supabase Storage
Deployment	Docker Compose, Render

Table 1. Technology stack

3.4 System Requirements

Software Requirements

- Node.js
- Next.js
- Expo
- Express.js
- PostgreSQL / Supabase
- Docker
- Modern web browser
- Internet connection

Hardware Requirements

- Smartphone or computer
- Minimum **4 GB RAM**
- Internet connectivity
- GPS/location access for mobile users

SYSTEM DESIGN

3.1 Database Design

The system uses PostgreSQL with Supabase to store user, farmer, transporter, vehicle, booking, trip, route and cost-sharing data. The database connects these entities to support the complete transport-sharing workflow.

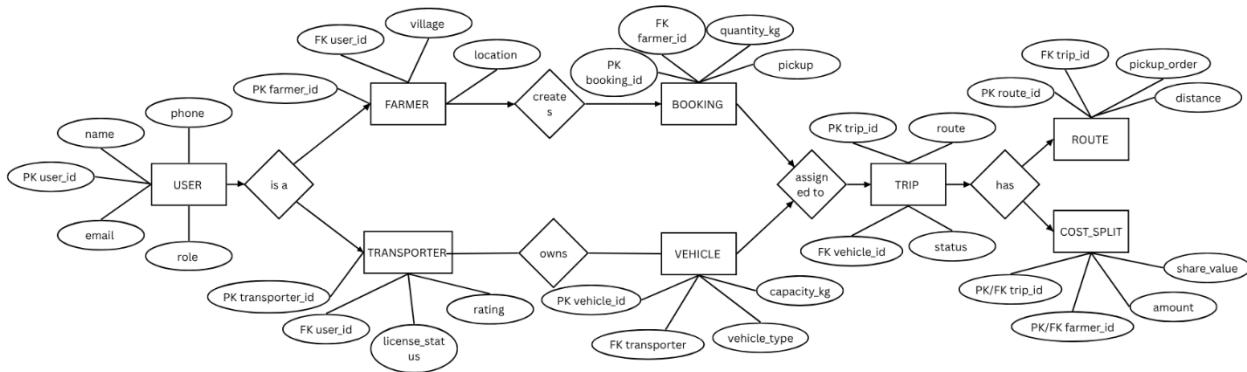


Figure 2. ER Diagram

3.2 User Roles and Access Control

There are three main roles in KisanPool.

Farmer – Makes transport requests, enters information regarding crops and quantities, and accesses bookings and the cost of shared transportation.

Transporter – Enters vehicle information, accesses appropriate transport requests and manages trips and deliveries.

Admin – Manages all users, vehicles, bookings and overall system.

Access to different features is granted by roles, meaning that a user cannot see anything except features that belong to his/her role.

3.3 APIs/Interfaces

APIs connect frontend and the backend built with Express.js framework.

Basic interfaces include: GET /api/vehicles/available

API	Purpose
/api/auth	User registration and login
/api/requests	Create and manage transport requests
/api/vehicles	Manage vehicle information
/api/matching	Match loads with vehicles
/api/routes	Generate and manage routes
/api/cost	Calculate shared transport cost
/api/bookings	Create and manage bookings

Table 2. APIs

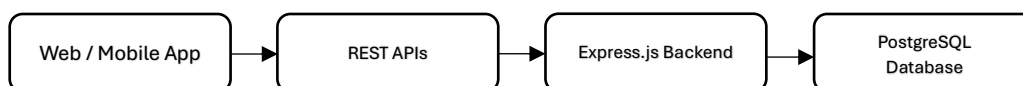


Figure 3. API Flow

TECHNICAL WORKFLOW & METHODOLOGY

4.1 End-to-End Workflow

KisanPool follows a simple process from creating a transport request to completing delivery:

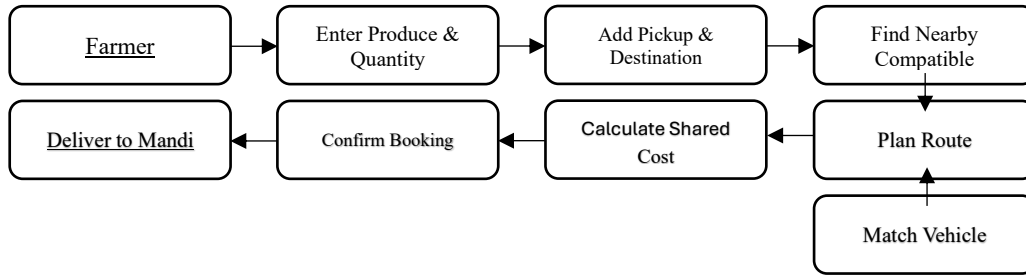


Figure 4. End-to-End Workflow

KisanPool takes care of the whole process, from generating a transport request to delivery at the mandi. The details of the produce are gathered, near-farmers having the same transport requirement are identified, a vehicle is booked, the pickup route is planned, the total cost for sharing is calculated, and then the delivery is made.

4.2 Flowchart

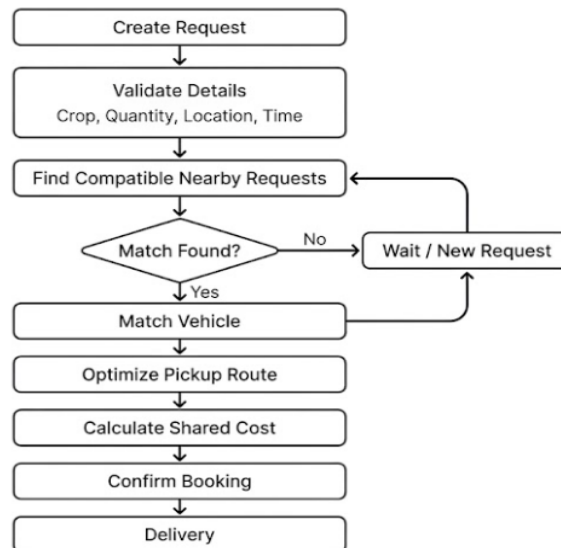


Figure 5. flowchart

First, KisanPool verifies the transportation requirement by the farmers and finds compatible transportation requirements, matching them with the appropriate vehicle, routing for pickup, and calculating the collective transportation expense before confirming their booking in mandi.

4.3 Core Logic / Algorithms

Load Matching: This process matches the pick-up location, destination, quantity, and timing requirement for transport.

Vehicle Matching: The appropriate vehicle is matched considering vehicle capacity, availability, and load requirement.

Route Optimization: Multiple pick-up locations are optimized before delivery at mandi.

Cost Allocation: Total cost of transportation is shared by the group of farmers traveling together.

4.4 Model / Methodology

KisanPool uses Request → Match → Optimize → Calculate → Deliver model.

Request: Gather farmer transport needs.

Match: Match compatible requests and vehicle.

Optimize: Optimize pickup routes.

Calculate: Calculate each farmer's cost.

Deliver: Deliver farmers to mandi.

IMPLEMENTATION

5.1 Module Based Implementation

The KisanPool platform has been designed based on some important modules that control the whole transport operation. There is an Authentication Module to control the login credentials of users and their different roles. The Transport Request Module helps in gathering details about crops, quantities, pickup and destination locations. The Vehicle Module handles transporter and vehicle data. The Matching Module helps in identifying matching farmers and appropriate vehicles. The Routing Module plans the optimal pickup routes.

5.2 Technology Stack

Component	Technology
Web & Mobile	Next.js, Expo
UI	Tailwind CSS, shaden/ui
Backend & API	Express.js, REST APIs
Database	PostgreSQL, Supabase
Authentication	Supabase Auth
Cache & Search	Upstash Redis, Meilisearch
File Storage	Supabase Storage
Deployment	Docker Compose, Render

Table 5. technology stack

5.2 Technical Components

The technical components include load matching, vehicle matching, route finding, booking, and cost sharing. The PostgreSQL database handles the application's data, and Supabase handles database, authentication, and storage services. The REST APIs serve to link web/mobility applications with the backend.

5.3 Implementation Details

The application uses role-based access for the farmers, transporters, and the administrator. The frontend applications use REST APIs to interact with the backend application which runs on Express.js. It processes the transport requests, matching, routing, and cost sharing and saves the data in the database.

TESTING & VALIDATION

6.1 Testing Approach

Testing of KisanPool is done on a module-by-module basis and also in totality. Major components which undergo testing include authentication of users, transport requests, vehicles, matching, routing, booking and costing. Integration testing tests the communication between the frontend, API, backend and database.

6.2 Test Cases

Test Case	Expected Result
User Login	User logs in successfully
Create Transport Request	Request is saved correctly
Add Vehicle	Vehicle details are stored
Find Match	Suitable vehicle/request is identified
Route Planning	Pickup route is generated
Cost Calculation	Shared cost is calculated
Confirm Booking	Booking status is updated

Table 6. test case

6.3 Validation Approach

The whole process is tested using farmer request and vehicle data samples. The system verifies if there was proper matching, route planning, booking, and if costs of transport are shared properly.

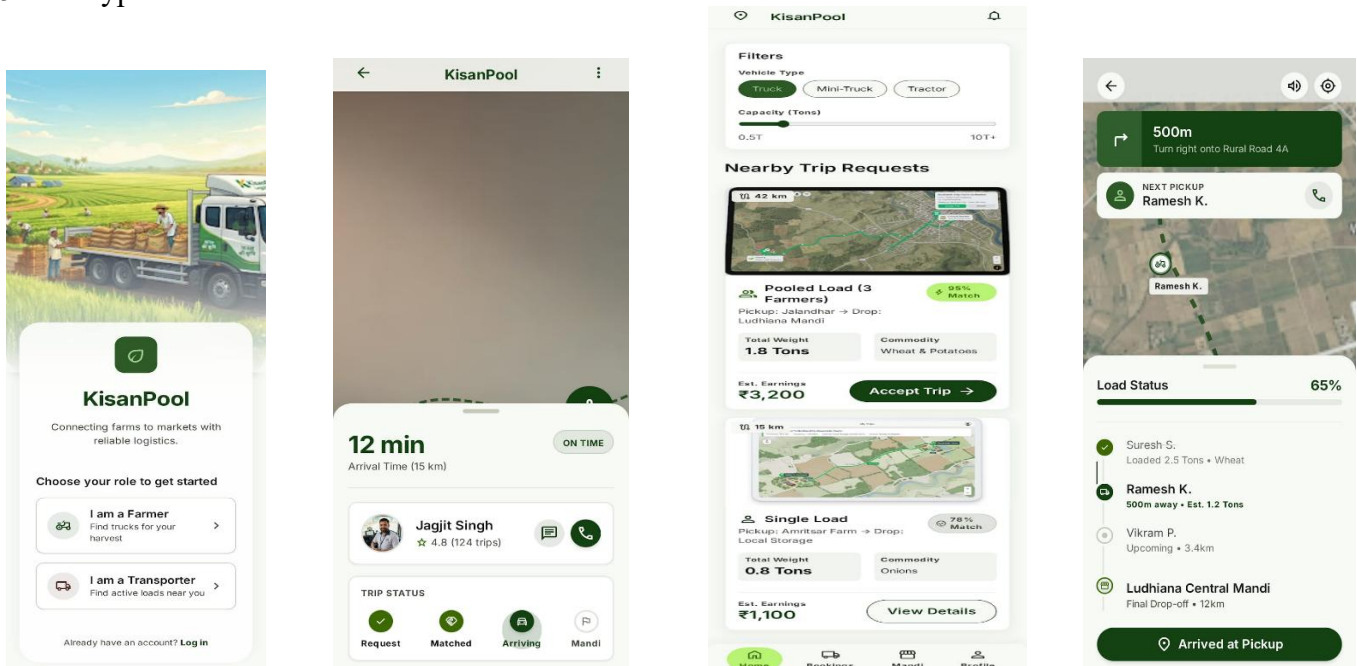
6.4 Performance / Evaluation Criteria

The following criteria may be used to evaluate the prototype:

- Matching Efficiency
- Distance/Travel Time
- Cost Saved for Farmers
- API Execution Time
- Booking Efficiency
- Error Rate

Use real-life measurements, if any, do not come up with percentages.

6.5 Prototype Screenshots



RESULTS, LIMITATIONS & FUTURE SCOPE

7.1 Results

The KisanPool prototype shows the full flow of shared transport process as follows:

- Farmer can make transport requests mentioning crop details, number and location of crops.
- Compatible transport requests nearby can be found for load sharing.
- Vehicles can be matched to the transport needs.
- Multi-stop pickup routes can be planned.
- Cost of transport can be distributed among the participating farmers.
- The entire process is managed from one place itself for requesting, booking vehicles.

KisanPool consolidates the transport requirements of many small farmers in one shared journey.

7.2 Expected Impact

Economic

- Lower shared transportation burden
- Better vehicle utilization
- Improved earning potential for farmers

Operational

- Fewer separate trips
- Coordinated pickup
- Better use of available vehicle capacity

Environmental

- Reduced duplicate transportation trips
- Better utilization of existing vehicles
- Potential reduction in fuel consumption

Social

- Easier access to transportation
- Transparent cost sharing
- Digital coordination for small farmers

7.3 Limitations

- Real-world testing requires more farmer and transporter data.
- Rural internet connectivity can affect usage.
- Route quality depends on available location data.
- Large-scale deployment requires further performance testing.
- Real-time market information depends on reliable external data.

7.4 Future Scope

- Real-time GPS vehicle tracking
- Demand prediction
- Advanced route optimization
- Agricultural market integration
- Multi-village and district-level deployment

REFERENCES

- [1] Government of India, “National Agriculture Market (e-NAM),” *Ministry of Agriculture & Farmers Welfare, Government of India*, 2026. <https://www.enam.gov.in/>
- [2] Government of India, “e-NAM Live Price Information,” *National Agriculture Market*, 2026. [e-NAM Live Price Information](#)
- [3] Government of India, “National Agriculture Market: Directory and Operational Information,” *Small Farmers’ Agribusiness Consortium (SFAC)*, 2021. [e-NAM Directory](#)
- [4] L. Bertazzi, M. Savelsbergh, and M. G. Speranza, “Inventory routing,” *Transportation Science*, vol. 36, no. 1, pp. 123–132, 2002.
- [5] G. Laporte, “The vehicle routing problem: An overview of exact and approximate algorithms,” *European Journal of Operational Research*, vol. 59, no. 3, pp. 345–358, 1992.
- [6] L. S. Shapley, “A value for n-person games,” in *Contributions to the Theory of Games II*, Princeton University Press, pp. 307–317, 1953.
- [7] M. W. P. Savelsbergh and M. Sol, “The general pickup and delivery problem,” *Transportation Science*, vol. 29, no. 1, pp. 17–29, 1995.

Technology References

- [8] Vercel, “Next.js Documentation,” *Next.js*, 2026. [Next.js Documentation](#)
- [9] Express.js, “Express.js Documentation,” *Express.js*, 2026. [Express.js Documentation](#)
- [10] Supabase, “Supabase Documentation,” *Supabase*, 2026. [Supabase Documentation](#)